

Clinic Management System

TECHNICAL REPORT



SUBMITTED BY

Benish Fatima (2024-ag-6641)

ADVISED BY

Sir Qamar Nawaz

A TECHNICAL REPORT SUBMITTED IN PARTIAL FULFILLMENT OF REQUIREMENT FOR
THE DEGREE OF

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCES

UNIVERSITY OF AGRICULTURE FAISALABAD

May 2026

DECLARATION

I hereby declare that the contents of the report "Clinic Management System" is project of my own and no part has been copied from any published source (except the references). I further declare that this work has not been submitted for award of any other diploma/degree. The university may take action if the information provided is found false at any stage. In case of any default the scholar will be proceeded against as per UAF policy.

Benish Fatima

2024-ag-6641

CERTIFICATE

To,
The Controller of Examinations,
University of Agriculture,
Faisalabad.

The supervisory committee certifies that Benish Fatima bearing AG # 2024-ag-6641 has successfully completed her project in partial fulfillment of requirement for the degree of B.Sc. Computer Science under our guidance and supervision.

Sir Qamar Nawaz
Department of Computer Science

Dr. Muhammad Ahsan Latif
Chairman,
Department of Computer Science

ACKNOWLEDGEMENT

I thank all who in one way or another contributed to the completion of this report. I am grateful to the Department of Computer Science, University of Agriculture Faisalabad for providing the opportunity to work on this project and for making it possible for me to study here.

My special and heartfelt thanks to my advisor, Sir Qamar Nawaz, whose guidance and supervision made this project possible. His teaching of Visual Programming concepts throughout the semester provided the foundation for building this application. His encouragement and direction brought this work to completion. For any faults, I take full responsibility.

I am also thankful to my fellow students whose feedback, suggestions, and collaborative spirit provided new ideas and helped improve the project throughout the development process.

Furthermore, I thank my family for their continuous encouragement, support, and prayers throughout this semester. May the Almighty God richly bless all of you.

ABSTRACT

This technical report presents the design and development of a Clinic Management System, a desktop application developed using C# Windows Forms with SQLite database integration. The system was developed to address the inefficiencies of manual paper-based clinic operations that are common in small medical facilities.

The application provides clinic staff with a centralized digital solution for managing medicine inventory, patient records, and billing operations. The system implements secure login using BCrypt password hashing, ensuring that administrator credentials are never stored as plain text in the database. BCrypt.Verify() is used for one-way password comparison during login.

The development followed a 3-tier architecture with the Repository pattern, clearly separating the presentation layer (Windows Forms), data access layer (Repository classes), and data layer (SQLite database). This architectural approach ensures clean code organization with zero SQL queries in form files. All database queries are written in dedicated Repository classes in the Database folder.

Key features include full CRUD operations on medicines and patients, an automated billing system that reduces medicine stock upon bill generation, a dashboard displaying live statistics including total medicines, patients, daily bills, revenue, and low stock alerts, and a reports module with color-coded inventory status showing green for in-stock, yellow for low stock, and red for out-of-stock medicines.

The system was developed using Visual Studio 2022, C# programming language with .NET Framework 4.7.2, Microsoft.Data.Sqlite NuGet package for database connectivity, and BCrypt.Net-Next for password security. Version control was maintained throughout development using Git with GitHub, with meaningful commits at each development milestone.

Testing confirmed that all CRUD operations function correctly, the billing system accurately calculates totals and reduces stock, and the login system properly validates credentials using BCrypt. The system successfully meets all minimum requirements specified for the Visual Programming CS-412 course project including CRUD on multiple entities, user login, search functionality, dashboard statistics, and report generation.

Table of Contents

CHAPTER 1 - INTRODUCTION.....	8
1.1 Background.....	9
1.2 Description.....	9
1.3 Problem Statement.....	10
1.4 Scope.....	10
1.5 Objectives	11
1.6 Feasibility.....	11
1.6.1 Technical Feasibility.....	12
1.6.2 Schedule Feasibility	12
1.6.3 Economic Feasibility	12
1.6.4 Operational Feasibility.....	12
1.6.5 Legal/Ethical Feasibility	12
1.6.6 Resource Feasibility.....	12
1.7 Requirements	12
1.7.1 Functional Requirements	12
1.7.2 Non-Functional Requirements	14
1.7.3 Hardware Requirements.....	14
1.7.4 Software Requirements.....	14
1.8 Stakeholders.....	14
CHAPTER 2 - MATERIALS & METHODS.....	16
2.1 Process Model.....	16
2.2 Tools & Technologies.....	16
2.3 Design	17
2.3.1 Use Case Diagram.....	17
2.3.2 Usage Scenarios	18
2.3.3 Sequence Diagram	20
2.3.4 Class Diagram.....	21
2.3.5 Data Flow Diagram.....	22
.....	23
2.3.6 ER Diagram	24
2.3.7 Database Model	24
2.3.8 Architecture.....	25
CHAPTER 3 - RESULTS & DISCUSSION	26

3.1 Testing.....	26
3.2 Test Cases	26
3.3 Conclusion	28
CHAPTER 4 - USER MANUAL	30
4.1 Starting the Application	30
4.2 Logging In.....	30
4.3 Dashboard	30
4.4 Medicine Inventory	31
4.5 Patient Management.....	33
4.6 Billing	34
4.7 Reports	35
4.8 Logging Out	36
REFERENCES	37

List of Figures

Figure 2.2 Use Case Diagram	17
Figure 2.3 Sequence Diagram.....	20
Figure 2.4 Class Diagram	21
Figure 2.5 Context Diagram	22
Figure 2.6 Level 0 DFD	22
Figure 2.7 Level 1 DFD	23
Figure 2.8 Entity Relationship Diagram	24
Figure 2.9 Database Model	24
Figure 2.10 Applications's Architecture	25
Figure 4.1 Signing in.....	30

CHAPTER 1 - INTRODUCTION

1.1 Background

Clinics and small medical facilities generate large volumes of data on a daily basis including medicine stock records, patient visit information, and financial billing data. In many small clinics across Pakistan and globally, this data continues to be managed manually through paper-based records and registers. This approach has existed for decades but has become increasingly inefficient as clinic operations grow in complexity and patient volumes increase.

Manual record keeping in clinics leads to several significant problems. Medicine stock levels are difficult to track accurately, leading to either overstocking of medicines that expire before use, or running out of essential medicines when patients need them most. Patient records are often disorganized and stored in physical files, making it slow and difficult to retrieve previous visit history or contact information. Billing processes conducted manually are slow and prone to calculation errors, leading to financial discrepancies.

The advancement of desktop application development technologies, particularly C# Windows Forms and lightweight databases like SQLite, has made it feasible to develop cost-effective management systems for small clinics. Such systems can run on standard computers without requiring expensive server infrastructure, dedicated database servers, or continuous internet connectivity. This makes them ideal for small clinics with limited technical resources.

This project was initiated to address these challenges by developing a Clinic Management System that digitalizes the core operations of a small clinic. The system provides an intuitive graphical user interface for managing medicines, patients, and billing from a single desktop application, completely eliminating the need for paper-based records and manual calculations. The project was developed as part of the Visual Programming (CS-412) course at the University of Agriculture Faisalabad, demonstrating practical application of Windows Forms development, database integration, and software architecture principles.

1.2 Description

The Clinic Management System is a desktop application designed for use by clinic administrative staff. The system provides a comprehensive digital solution for the three core operational areas of a small clinic: medicine inventory management, patient record management, and billing.

The application is built using C# Windows Forms, a mature and widely-used framework for building desktop GUI applications on the Windows platform. The user interface consists of six forms: a login form for secure access, a dashboard for navigation and statistics, and dedicated management forms for medicines, patients, billing, and reports. Each form follows a consistent visual design with a dark blue title panel and color-coded action buttons.

The database layer uses SQLite, a serverless file-based database system. The entire database is stored in a single file called clinic.db which is automatically created when the application runs for the first time through the DatabaseHelper.InitializeDatabase() method called from Program.cs. This eliminates the need for any manual database setup, server installation, or configuration, making the system deployable on any standard Windows computer.

The system implements a 3-tier architecture following the Repository pattern. All SQL queries are written exclusively in dedicated Repository classes within the Database folder: UserRepository, MedicineRepository, PatientRepository, and BillRepository. Form files contain only UI logic and call repository methods by passing and receiving Model objects. Model classes in the Models folder serve as data carriers between layers. This architectural approach follows industry best practices for separation of concerns and code maintainability.

Security is implemented through BCrypt password hashing using the BCrypt.Net-Next NuGet package. The administrator password is never stored as plain text in the database. BCrypt converts the password to a one-way hash that cannot be decrypted. During login, BCrypt.Verify() performs a safe comparison between the entered plain text password and the stored hash without any decryption. A key feature is automatic stock management where medicine quantities are automatically reduced when bills are generated.

1.3 Problem Statement

Small clinics currently rely on manual paper-based systems to manage medicine inventory, patient records, and billing operations. This results in inaccurate stock tracking leading to medicine shortages or overstocking, slow and error-prone manual billing calculations, difficulty retrieving patient records for follow-up visits, no real-time visibility into clinic statistics and revenue, and risk of data loss due to physical damage to paper records. There is a need for a digital desktop application that automates these operations, reduces human error, and provides clinic staff with accurate real-time information from a single interface without requiring any server installation or complex setup.

1.4 Scope

The Clinic Management System covers the following scope:

In Scope:

- Single administrator login with BCrypt secure password authentication
- Complete medicine inventory management with stock quantity tracking
- Patient record management with search and filter functionality
- Billing system with automatic stock reduction upon bill generation
- Dashboard with live statistics updated on each login
- Reports with billing history search and inventory color-coded status
- Data persistence using SQLite database auto-created on first run

Out of Scope:

- Multi-user or role-based access control
- Network or web-based access
- Integration with external pharmacy or hospital systems
- Appointment scheduling or prescription management
- Financial accounting beyond basic billing totals

1.5 Objectives

Goal: To develop a desktop application that digitalizes clinic operations and eliminates manual paper-based record keeping.

Objectives:

1. Develop a secure login system using BCrypt password hashing
2. Implement full CRUD operations for medicine inventory management with stock tracking
3. Implement full CRUD operations for patient record management
4. Develop an automated billing system that calculates totals and reduces medicine stock automatically
5. Create a dashboard showing live clinic statistics on login
6. Generate reports for billing history and inventory status with color coding
7. Implement 3-tier architecture with Repository pattern for clean code separation
8. Use SQLite for serverless, portable, automatically-initialized data storage

1.6 Feasibility

1.6.1 Technical Feasibility

The project uses C# Windows Forms, a well-established technology with extensive documentation and support. SQLite is a proven database system used in millions of applications worldwide. All required knowledge of C#, OOP, database design and Windows Forms development was gained through the CS-412 course. All required tools including Visual Studio 2022 Community Edition are freely available.

1.6.2 Schedule Feasibility

The project was completed within the semester timeline of approximately eight weeks. Development was structured into phases: database design and login, medicine and patient management, billing system, reports and dashboard statistics, architecture refactoring with Repository pattern, and final testing and documentation.

1.6.3 Economic Feasibility

All tools and technologies used are completely free. Visual Studio Community Edition, SQLite, Microsoft.Data.Sqlite, and BCrypt.Net-Next packages are all available at no cost. The application runs on standard Windows computers without requiring any specialized or expensive hardware.

1.6.4 Operational Feasibility

The application has an intuitive graphical interface with clearly labeled buttons and consistent navigation. Clinic staff with basic computer knowledge can learn to use the system quickly. No technical expertise is required to operate the system after initial setup.

1.6.5 Legal/Ethical Feasibility

The application uses only open-source technologies with appropriate licenses. Patient data is stored locally on the clinic's own computer, maintaining complete data privacy. No external data transmission occurs and no internet connection is required for operation.

1.6.6 Resource Feasibility

Development required one developer, Visual Studio 2022 Community, and a standard Windows laptop. No additional hardware, software licenses, or financial resources were required.

1.7 Requirements

1.7.1 Functional Requirements

FR01: User Login

- FR01-01: System shall display login form on application startup
- FR01-02: System shall collect username and password from user
- FR01-03: System shall verify credentials against database using BCrypt.Verify()
- FR01-04: System shall open Dashboard on successful login
- FR01-05: System shall display error message on invalid credentials
- FR01-06: System shall prevent empty field submission with warning message

FR02: Medicine Management

- FR02-01: System shall allow admin to add new medicine records
- FR02-02: System shall display all medicines in DataGridView with search
- FR02-03: System shall allow admin to update existing medicine information
- FR02-04: System shall allow admin to delete medicines without billing history
- FR02-05: System shall prevent deletion of medicines used in billing

FR03: Patient Management

- FR03-01: System shall allow admin to add new patient records
- FR03-02: System shall display all patients in DataGridView with search
- FR03-03: System shall allow admin to update existing patient information
- FR03-04: System shall allow admin to delete patients without billing history
- FR03-05: System shall prevent deletion of patients with existing bills

FR04: Billing

- FR04-01: System shall allow selection of registered patient from dropdown
- FR04-02: System shall allow selection of in-stock medicines from dropdown
- FR04-03: System shall check stock availability before adding to bill
- FR04-04: System shall calculate total bill amount automatically
- FR04-05: System shall save bill record to database on confirmation
- FR04-06: System shall reduce medicine stock quantity after bill generation

FR05: Reports

- FR05-01: System shall display all bills with patient names using JOIN query
- FR05-02: System shall display total revenue using SUM aggregation
- FR05-03: System shall display inventory with color-coded stock status

FR06: Dashboard

Clinic Management System

- FR06-01: System shall display live count of registered medicines
- FR06-02: System shall display live count of registered patients
- FR06-03: System shall display count of bills generated today
- FR06-04: System shall display total revenue from all bills
- FR06-05: System shall display low stock alert count (medicines < 10 units)

1.7.2 Non-Functional Requirements

- NFR01: System shall start and display dashboard within 3 seconds on standard hardware
- NFR02: System shall support single admin user login
- NFR03: Passwords shall be stored as BCrypt hash, never as plain text
- NFR04: Database shall be created automatically on first run without manual setup
- NFR05: System shall run on Windows 10 or later
- NFR06: System shall provide confirmation dialogs before all delete operations
- NFR07: System shall validate all input fields before executing database operations
- NFR08: System shall display friendly error messages instead of technical exceptions

1.7.3 Hardware Requirements

- Processor: Intel Core i3 or equivalent or higher
- RAM: 2GB minimum, 4GB recommended
- Hard Disk: 100MB free space minimum
- Display: 1024x768 resolution minimum
- Input: Standard keyboard and mouse

1.7.4 Software Requirements

- Operating System: Windows 10 or later
- Framework: .NET Framework 4.7.2 (pre-installed on Windows 10+)
- Database: SQLite (embedded, no separate installation required)
- Development: Visual Studio 2022 (for source code compilation)

1.8 Stakeholders

Clinic Administrator: Primary user of the system. Manages medicines, patients and billing on daily basis. Direct beneficiary of the system's efficiency improvements.

Clinic Owner/Manager: Reviews reports and revenue statistics to monitor clinic performance. Uses dashboard for quick overview of daily operations.

Patients: Indirectly affected by the system. Their records and bills are managed more accurately and efficiently through the system, resulting in faster service.

Development Team: Student developer (Benish Fatima) responsible for design, development, testing and documentation of the system.

Course Instructor: Sir Qamar Nawaz, Department of Computer Science, UAF. Evaluates the project against Visual Programming CS-412 course requirements.

CHAPTER 2 - MATERIALS & METHODS

2.1 Process Model

The Agile development model was followed for this project. Agile is a Rapid Application Development model that is a type of incremental model. Development was divided into short phases where each form and feature was developed, tested, and verified before moving to the next phase. This allowed early detection and resolution of issues at each stage rather than discovering problems after the entire system was built.

Phase 1: Project setup, folder structure creation, database design, and login form with BCrypt authentication.

Phase 2: Medicine Inventory form with full CRUD and search functionality.

Phase 3: Patient Management form with full CRUD and deletion protection.

Phase 4: Billing system with stock management and automatic total calculation.

Phase 5: Reports form with billing history and inventory status.

Phase 6: Dashboard with live statistics and Repository pattern refactoring.

Phase 7: Final testing, bug fixing, Git commits, and documentation.

The incremental Agile approach was particularly beneficial for this project. For example, namespace errors caused by moving forms into the Forms folder for proper architecture were caught and resolved in Phase 1 before they could affect later phases. The billing scope error was caught during Phase 4 testing before integration with reports in Phase 5.

2.2 Tools & Technologies

- Visual Studio 2022 Community Edition: Integrated development environment for C# development and Windows Forms designer
- C# (.NET Framework 4.7.2): Primary object-oriented programming language
- Windows Forms: Microsoft UI framework for desktop GUI application development
- SQLite: Serverless, file-based, embedded relational database system
- Microsoft.Data.Sqlite (NuGet Package): SQLite database connectivity library for .NET
- BCrypt.Net-Next (NuGet Package): Password hashing library implementing BCrypt algorithm
- Git: Distributed version control system for tracking code changes
- GitHub: Cloud-based Git repository hosting and collaboration platform

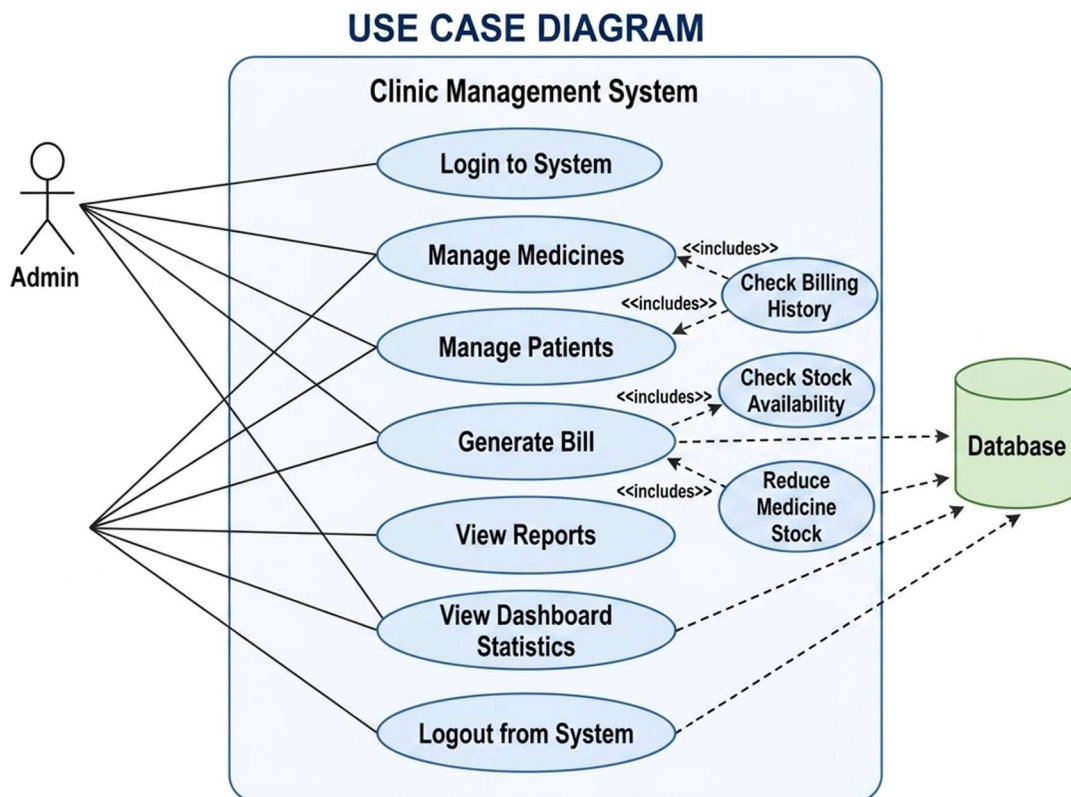
- DB Browser for SQLite: Desktop tool for viewing and verifying SQLite database contents during development

2.3 Design

2.3.1 Use Case Diagram

The Clinic Management System has one actor: the Admin. The Admin interacts with all use cases in the system. Below are the identified use cases:

- Login to System
- Manage Medicines (Add Medicine, Update Medicine, Delete Medicine, Search Medicines)
- Manage Patients (Add Patient, Update Patient, Delete Patient, Search Patients)
- Generate Bill
- View Reports
- View Dashboard Statistics
- Logout from System



2.3.2 Usage Scenarios

Table 2.1: Login to System

Use Case Title	Login to System
Use Case Id	1
Requirement Id	FR01
Description	Admin logs in to the Clinic Management System using username and password verified through BCrypt hashing.
Pre Conditions	Application is running. Database with admin account exists.
Task Sequence	1. Admin enters username in txtUsername field. 2. Admin enters password in txtPassword field. 3. Admin clicks Login button or presses Enter. 4. System calls UserRepository.ValidateLogin(). 5. BCrypt.Verify() compares entered password with stored hash. 6. Dashboard opens on successful verification.
Exceptions	Invalid credentials: error message displayed. Empty fields: warning message displayed. Database error: error message with details shown.
Post Conditions	Dashboard form is displayed. Admin is logged in.
Authority	Admin
Modification History	1.0
Author	Benish Fatima

Table 2.2: Add Medicine

Use Case Title	Add Medicine
Use Case Id	2
Requirement Id	FR02
Description	Admin adds a new medicine record to the inventory database.
Pre Conditions	Admin is logged in. Medicine Inventory form is open.
Task Sequence	1. Admin enters medicine name in txtName. 2. Admin enters quantity in txtQty. 3. Admin enters price in txtPrice. 4. Admin enters expiry date in txtExpiry. 5. Admin clicks Add button. 6. System calls MedicineRepository.AddMedicine(). 7. Grid refreshes showing new medicine.
Exceptions	Empty required fields: validation warning shown, insertion cancelled. Duplicate medicine name: database error shown.

Post Conditions	New medicine record saved in Medicines table. Medicine appears in DataGridView.
Authority	Admin
Modification History	1.0
Author	Benish Fatima

Table 2.3: Generate Bill

Use Case Title	Generate Bill
Use Case Id	3
Requirement Id	FR04
Description	Admin generates a bill for a patient by selecting medicines and quantities.
Pre Conditions	Admin is logged in. At least one patient and one in-stock medicine exist.
Task Sequence	1. Admin opens Billing form. 2. Admin selects patient from cmbPatient dropdown. 3. Admin selects medicine from cmbMedicine dropdown. 4. Admin enters quantity in txtQty. 5. Admin clicks Add Item. 6. System checks stock via MedicineRepository.GetMedicineStock(). 7. Admin repeats steps 3-6 for more medicines. 8. Admin clicks Generate Bill. 9. System calls BillRepository.GenerateBill(). 10. Bill saved, stock reduced automatically.
Exceptions	Insufficient stock: warning message shown, item not added. No patient selected: warning message shown. Empty bill: warning message shown.
Post Conditions	Bill record saved in Bills table. BillItems saved in BillItems table. Medicine quantities reduced in Medicines table.
Authority	Admin
Modification History	1.0
Author	Benish Fatima

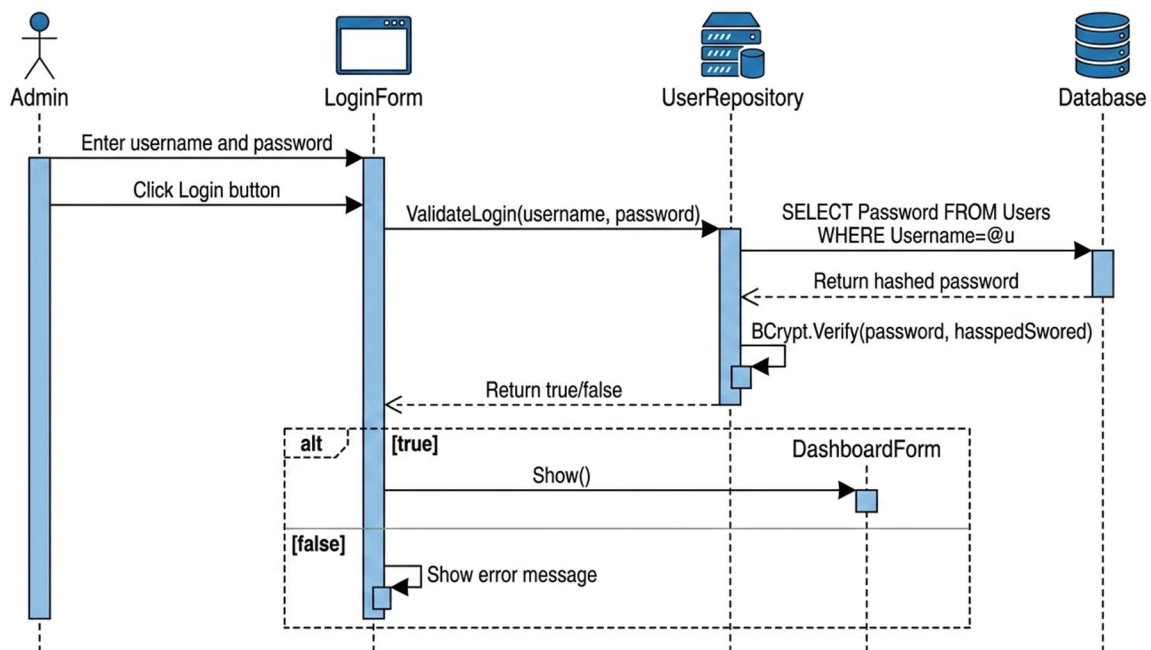
Table 2.4: Delete Patient

Use Case Title	Delete Patient
Use Case Id	4
Requirement Id	FR03
Description	Admin deletes a patient record from the system.

Pre Conditions	Admin is logged in. Patient Management form is open.
Task Sequence	1. Admin clicks on patient row in DataGridView. 2. Fields fill automatically. 3. Admin clicks Delete button. 4. Confirmation dialog appears. 5. Admin clicks Yes. 6. System calls PatientRepository.DeletePatient(). 7. Repository checks COUNT of bills for this patient. 8. If no bills, patient is deleted and grid refreshes.
Exceptions	Patient has billing history: warning message shown, deletion cancelled. No patient selected: warning message shown.
Post Conditions	Patient record removed from Patients table if no billing history.
Authority	Admin
Modification History	1.0
Author	Benish Fatima

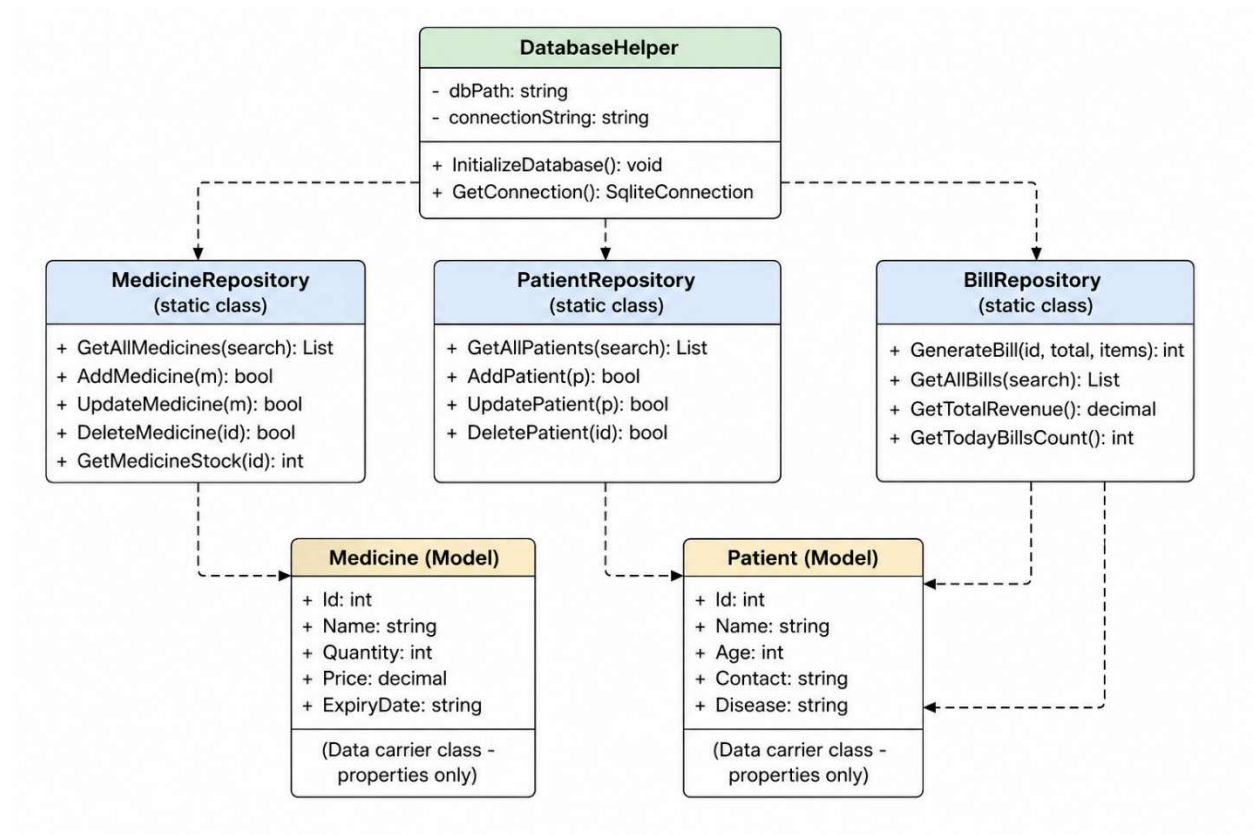
2.3.3 Sequence Diagram

Sequence Diagram - Login Process:



2.3.4 Class Diagram

Main classes in the Clinic Management System:



2.3.5 Data Flow Diagram

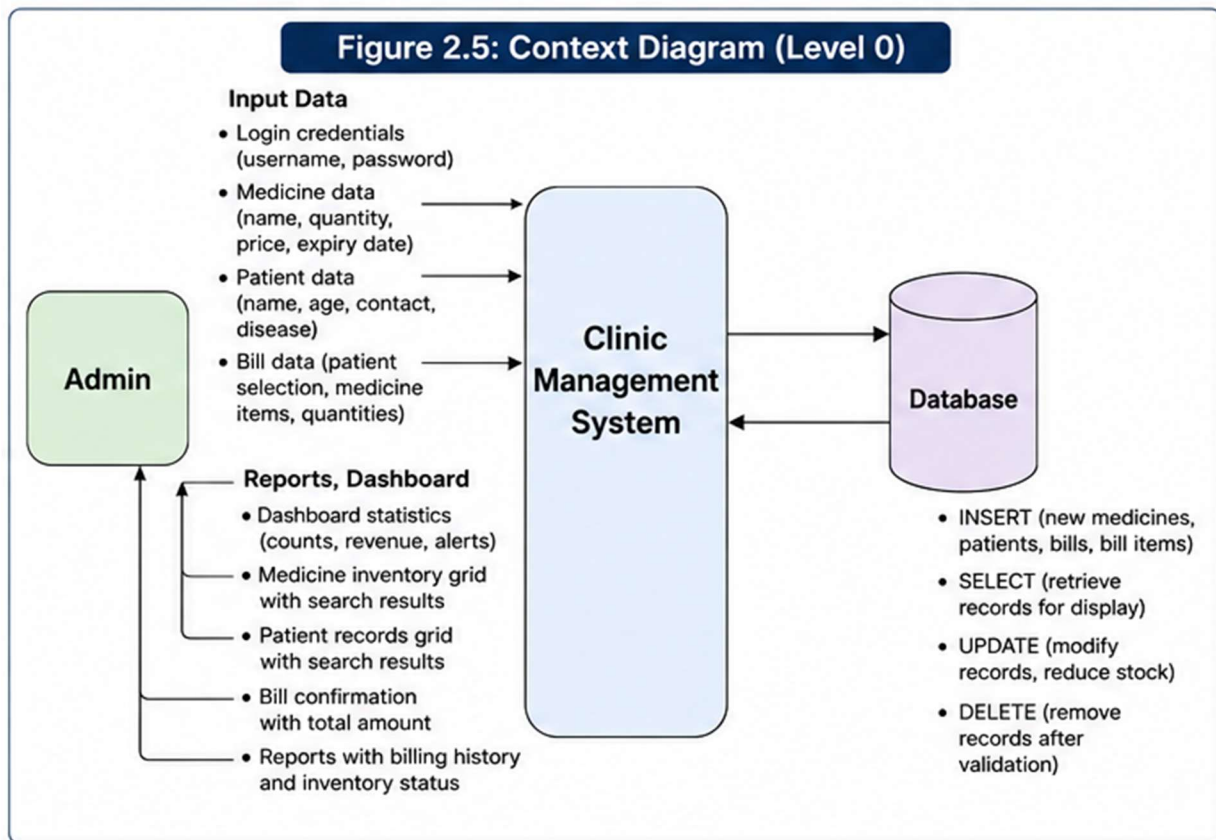


Figure 2.6: Level 0 DFD

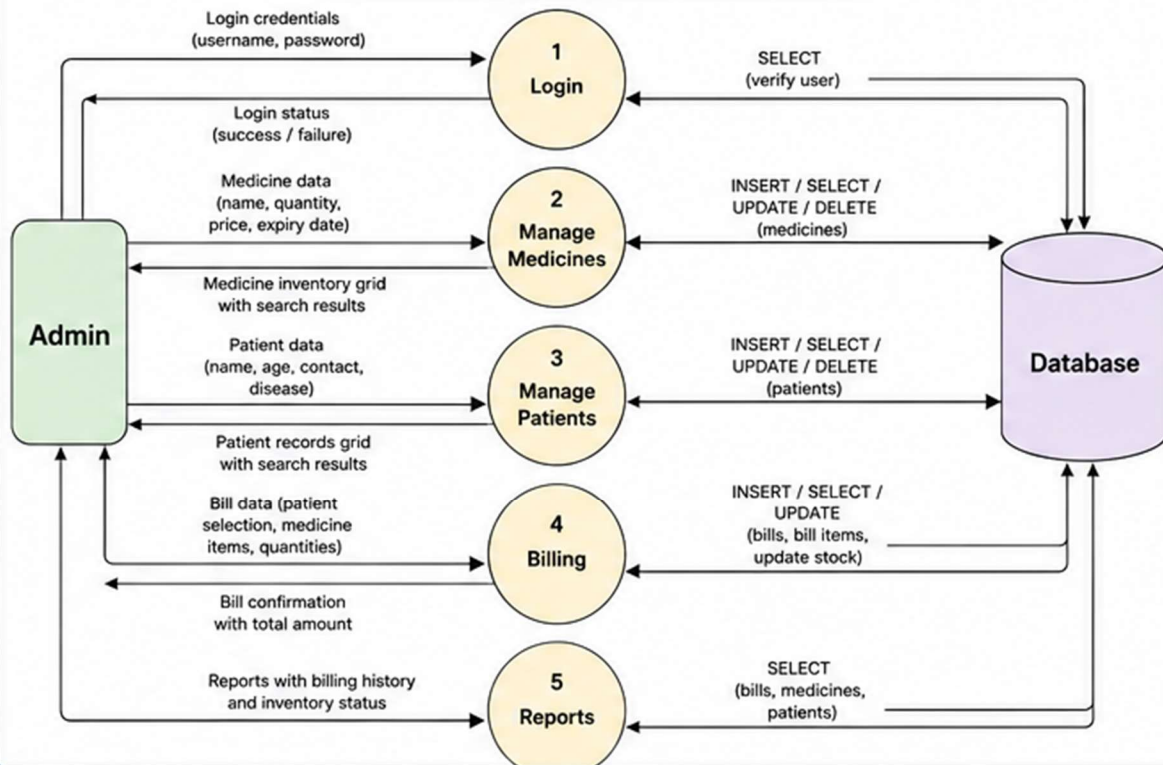
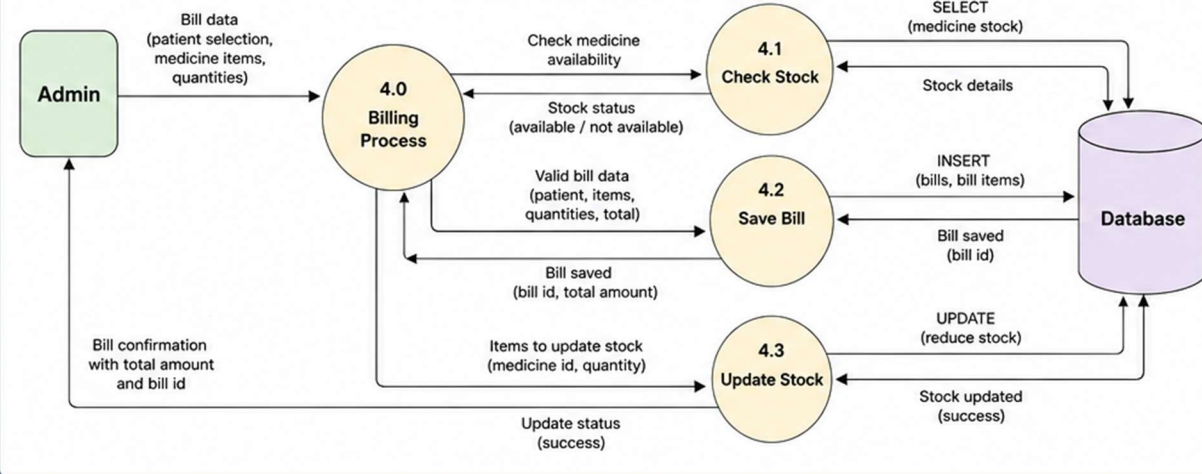
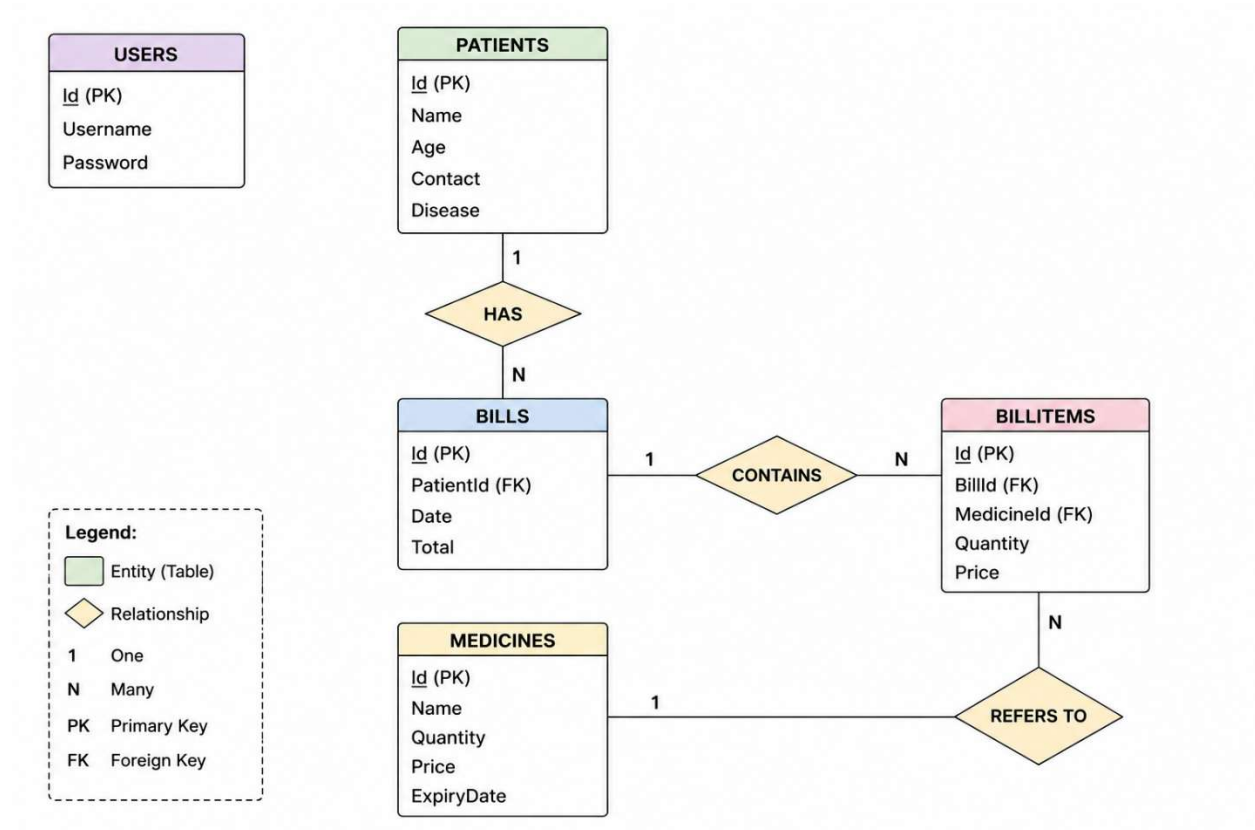


Figure 2.7: Level 1 DFD – Billing



2.3.6 ER Diagram

The Entity Relationship Diagram shows the following entities and relationships:



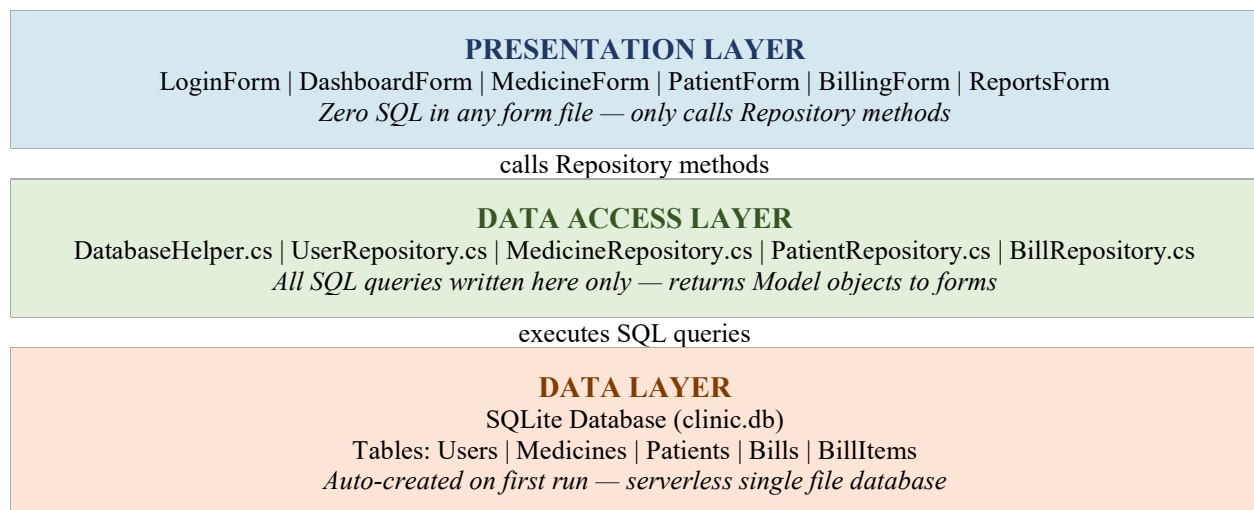
2.3.7 Database Model

Table	Field	Type	Description
Users	Id	INTEGER	Primary Key, AutoIncrement
Users	Username	TEXT	Not Null, Unique
Users	Password	TEXT	BCrypt Hashed, Not Null
Medicines	Id	INTEGER	Primary Key, AutoIncrement
Medicines	Name	TEXT	Medicine name, Not Null
Medicines	Quantity	INTEGER	Current stock quantity
Medicines	Price	REAL	Price per unit
Medicines	ExpiryDate	TEXT	Format: yyyy-MM-dd
Patients	Id	INTEGER	Primary Key, AutoIncrement
Patients	Name	TEXT	Patient name, Not Null
Patients	Age	INTEGER	Patient age

Patients	Contact	TEXT	Phone number
Patients	Disease	TEXT	Diagnosis/condition
Bills	Id	INTEGER	Primary Key, AutoIncrement
Bills	PatientId	INTEGER	Foreign Key → Patients.Id
Bills	Date	TEXT	Bill date yyyy-MM-dd
Bills	Total	REAL	Total bill amount
BillItems	Id	INTEGER	Primary Key, AutoIncrement
BillItems	BillId	INTEGER	Foreign Key → Bills.Id
BillItems	MedicineId	INTEGER	Foreign Key → Medicines.Id
BillItems	Quantity	INTEGER	Quantity sold
BillItems	Price	REAL	Price at time of sale

2.3.8 Architecture

The Clinic Management System follows a 3-Tier Architecture with Repository Pattern:



Models Folder: Medicine.cs, Patient.cs, Bill.cs — simple data carrier classes used to pass data between layers without exposing database logic to forms.

Key Benefits of This Architecture:

- Separation of Concerns: Each layer has exactly one responsibility
- Maintainability: SQL query changes only require updating one Repository file
- Readability: Forms contain clean UI code, Repositories contain clean database code
- Extensibility: New features can be added by creating new Repository methods

CHAPTER 3 - RESULTS & DISCUSSION

3.1 Testing

The Clinic Management System was tested using functional testing methodology. Each feature was tested against its corresponding functional requirement to verify correct behavior. Both positive tests using valid inputs and negative tests using invalid inputs and edge cases were performed systematically.

All CRUD operations were individually tested for each entity including medicines, patients, and bills. The billing system was tested end-to-end with multiple medicine items and stock reduction was verified after each bill generation. The login system was tested with valid credentials, invalid credentials, and empty field scenarios. Input validation was tested by intentionally leaving required fields empty. Deletion protection was verified by attempting to delete patients and medicines with existing billing records.

3.2 Test Cases

Table 3.1: User Login Test Case

Test Case ID	TC-01
Test Case Title	Verify Login Functionality
Test Case Priority	High
Requirement	FR01 - User Login
Test Description	Test the complete login process with valid, invalid and empty credentials
Pre-Conditions	Application is running. Admin account exists in database with BCrypt hashed password.
Test Steps	1. Enter username: admin, password: 1234, click Login 2. Enter username: admin, password: wrong, click Login 3. Leave username empty, click Login 4. Leave password empty, click Login
Expected Results	1. Dashboard form opens successfully 2. Error message: Invalid username or password 3. Warning: Please enter username and password 4. Warning: Please enter username and password
Actual Results	All results as expected
Status (Pass/Fail)	Pass

Table 3.2: Add Medicine Test Case

Test Case ID	TC-02
Test Case Title	Verify Add Medicine Functionality
Test Case Priority	High
Requirement	FR02 - Medicine Management
Test Description	Test adding a new medicine record with valid and invalid inputs
Pre-Conditions	Admin is logged in. Medicine Inventory form is open.
Test Steps	1. Enter valid name, quantity, price, expiry date, click Add 2. Leave medicine name empty, click Add 3. Leave quantity empty, click Add 4. Enter negative price, click Add
Expected Results	1. Medicine appears in DataGridView 2. Warning: Please enter medicine name 3. Warning: Please enter quantity 4. Warning message for invalid price
Actual Results	All results as expected
Status (Pass/Fail)	Pass

Table 3.3: Generate Bill Test Case

Test Case ID	TC-03
Test Case Title	Verify Bill Generation and Stock Reduction
Test Case Priority	High
Requirement	FR04 - Billing
Test Description	Test complete billing process including automatic stock reduction
Pre-Conditions	Admin logged in. Patient Ali exists. Medicine Novidate with quantity 10 exists.
Test Steps	1. Open Billing, select Ali, select Novidate, enter qty 2, click Add Item 2. Click Generate Bill, click Yes on confirmation 3. Open Medicine Inventory, check Novidate quantity
Expected Results	1. Novidate appears in bill grid with subtotal 2. Success message with bill ID and total 3. Novidate quantity reduced from 10 to 8
Actual Results	All results as expected
Status (Pass/Fail)	Pass

Table 3.4: Deletion Protection Test Case

Test Case ID	TC-04
Test Case Title	Verify Deletion Protection for Patient with Bills
Test Case Priority	Medium
Requirement	FR03 - Patient Management
Test Description	Test that a patient with existing billing records cannot be deleted
Pre-Conditions	Admin logged in. Patient Ali has at least one bill in the system.
Test Steps	1. Open Patient Management 2. Click on Ali in the DataGridView 3. Click Delete button 4. Click Yes on confirmation dialog
Expected Results	Warning message: Cannot delete this patient! This patient has X bill(s) on record. Patient remains in the system.
Actual Results	Warning shown as expected. Patient not deleted.
Status (Pass/Fail)	Pass

3.3 Conclusion

The Clinic Management System was successfully developed, tested, and verified against all functional and non-functional requirements. All four test cases passed with results exactly as expected. The login system correctly validates credentials using BCrypt one-way comparison. The billing system accurately calculates totals and reduces medicine stock automatically. Deletion protection correctly prevents removal of records with billing history, maintaining data integrity through foreign key relationships.

From a user interface and user experience perspective, the system provides a clean, consistent, and professional interface across all six forms. A unified color scheme with dark blue title panels and color-coded buttons provides intuitive visual cues: green for add operations, blue for update, red for delete, and grey for clear. The DataGridView component on each form provides clear tabular display of all records with live search filtering that updates results as the user types.

The dashboard provides immediate operational value by showing the current state of the clinic upon login: medicine count, patient count, today's bills, total revenue, and low stock alerts. The low stock counter turns red when any medicine falls below ten units, providing an immediate visual warning to clinic staff without requiring them to check the inventory form separately.

The Repository pattern architecture ensures a clean and maintainable codebase. Each form contains zero SQL queries and only interacts with the database through Repository method calls passing and receiving Model objects. This makes the code easy to read, debug, and extend. The DatabaseHelper.InitializeDatabase() method called from Program.cs ensures the system is fully self-configuring on any new machine with no manual setup required.

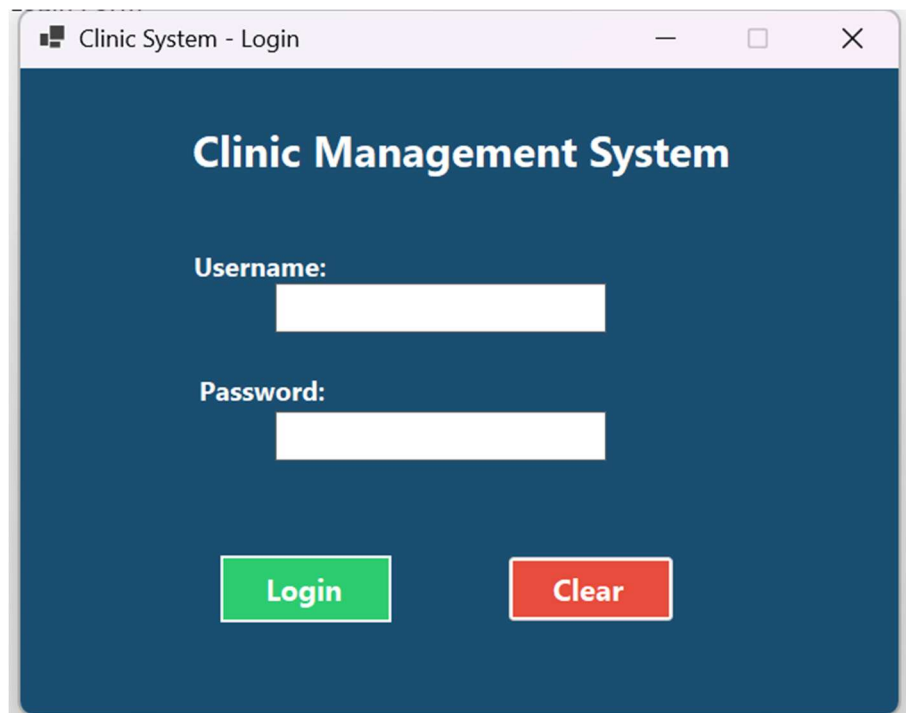
The system meets and exceeds all minimum project requirements for the Visual Programming CS-412 course: CRUD on multiple entities, user login with BCrypt security, live search functionality, dashboard with statistics, and report generation. Additional beyond-minimum features include automatic stock management, deletion protection with friendly warnings, color-coded inventory status, and proper 3-tier architecture with Repository pattern. The system is deployment-ready on any standard Windows computer with .NET Framework 4.7.2.

CHAPTER 4 - USER MANUAL

This chapter provides step-by-step instructions for using the Clinic Management System. The system is designed for clinic administrative staff with basic computer knowledge.

4.1 Starting the Application

Open the project in Visual Studio 2022 and press F5, or run the executable from the bin/Debug folder. The Login form appears automatically. The database clinic.db is created automatically on first run with all required tables and the default admin account.



4.2 Logging In

Enter the following credentials on the Login form:

- Username: admin
- Password: 1234

Click the Login button or press the Enter key. The system verifies credentials using BCrypt. If correct, the Dashboard opens. If incorrect, an error message is shown. If fields are empty, a warning message is shown.

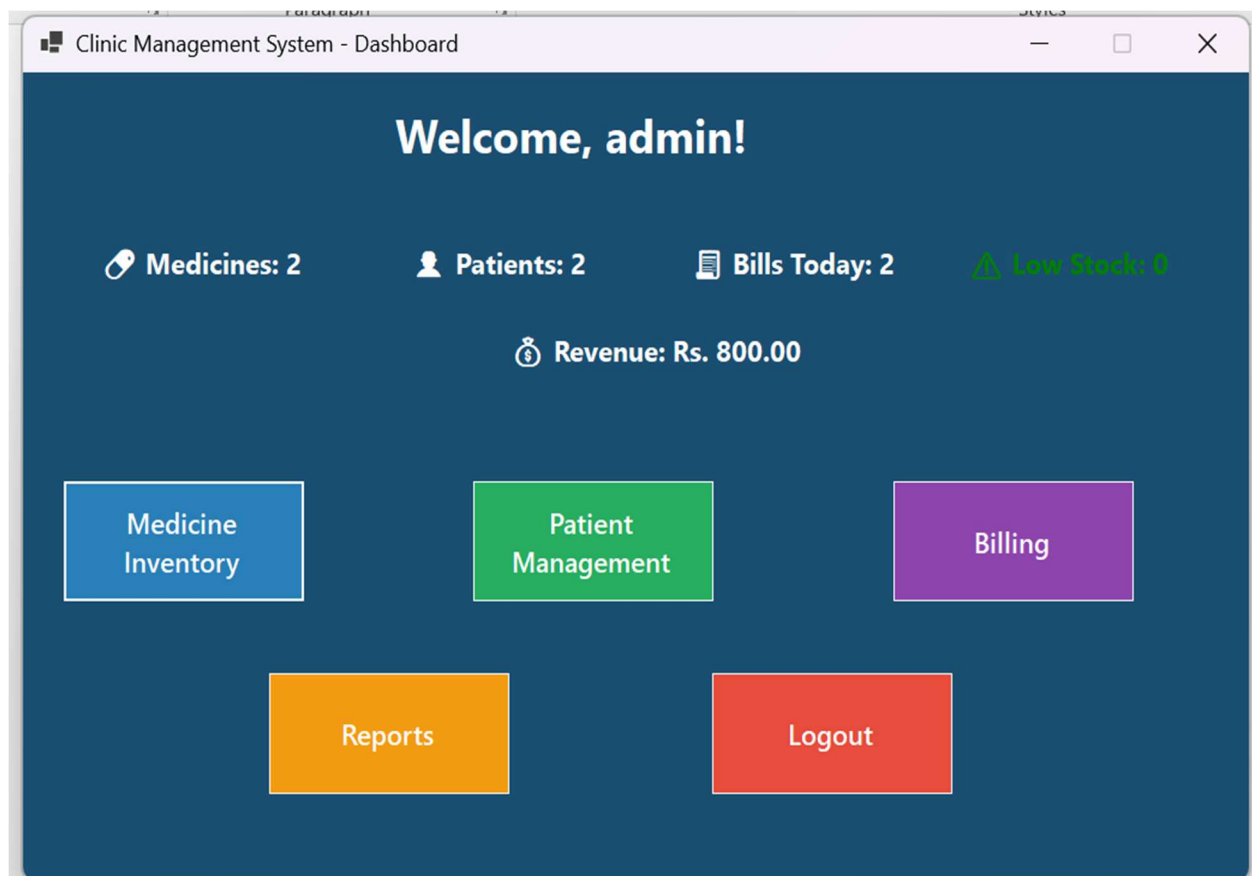
4.3 Dashboard

After successful login, the Dashboard opens showing live statistics at the top:

Clinic Management System

- Medicines count: total registered medicines
- Patients count: total registered patients
- Bills Today: number of bills generated today
- Revenue: total revenue from all bills
- Low Stock: count of medicines with quantity below 10 (shown in red if any)

Click any button to navigate to the corresponding management form. Click Logout to return to the Login form.



4.4 Medicine Inventory

Click Medicine Inventory on the Dashboard to open this form.

Adding a Medicine:

9. Enter Medicine Name in the text field

10. Enter Quantity (current stock)
11. Enter Price per unit
12. Enter Expiry Date (e.g. 2026-12-31)
13. Click Add — medicine appears in the grid below

Updating a Medicine:

14. Click any row in the DataGridView — fields fill automatically
15. Make the required changes in the fields
16. Click Update — changes saved to database

Deleting a Medicine:

17. Click the medicine row to select it
18. Click Delete — confirmation dialog appears
19. Click Yes to confirm deletion

Note: Cannot delete medicines that have been used in billing history.

Searching Medicines:

Type in the Search box — the DataGridView filters live as you type, showing only medicines whose names contain the typed text.

Medicine Inventory

Medicine Inventory

Search:

Medicine Name:

Quantity:

Price:

Expiry Date:

Add

Update

Delete

Clear

Medicine Name	Quantity	Price (Rs.)	Expiry Date
Paracetamol	42	100.00	12-07-2026
Creasat	30	300.00	15-08-2026

4.5 Patient Management

Click Patient Management on the Dashboard to open this form. Operations work identically to Medicine Inventory:

- Add Patient: Enter Name, Age, Contact, Disease — click Add Patient
- Update: Click row to select, modify fields, click Update
- Delete: Click row, click Delete, confirm (cannot delete patient with billing history)
- Search: Type in Search box to filter by patient name

- Click Clear to reset the entire bill form

Generate Bill

Select Patient: -- Select Patient -- ▾

Select Medicine: -- Select Medicine -- ▾

Quantity:

Total Amount: **Rs. 0.00**

Add Item

Remove Item

Generate Bill

Clear

Medicine	Quantity	Unit Price	Subtotal
----------	----------	------------	----------

4.7 Reports

Click Reports on the Dashboard to open the reports form.

Billing Report Tab:

- Shows all generated bills with patient names, dates and totals
- Total revenue displayed at bottom
- Search by patient name using the search box
- Click Refresh to update the report

Inventory Report Tab:

- Shows all medicines with current stock levels
- Color coding: Red = Out of Stock (Quantity = 0)
- Color coding: Yellow = Low Stock (Quantity less than 10)

- Reports

—□×

Reports

Billing ReportInventory Report

Search by Patient:

Refresh

Total Revenue: Rs. 800.00

Bill ID	Patient Name	Date	Total (Rs.)
1	Fatima	2026-05-21	Rs. 300.00
2	Benish	2026-05-21	Rs. 500.00

Click the Logout button on the Dashboard. A confirmation dialog appears asking Are you sure you want to logout? Click Yes to confirm. The system closes the Dashboard and returns to the Login form.

REFERENCES

- [1] Microsoft Corporation. (2024). Windows Forms Overview - .NET Framework. Microsoft Documentation. Retrieved from <https://docs.microsoft.com/en-us/dotnet/desktop/winforms/>

- [2] SQLite Consortium. (2024). SQLite Documentation - About SQLite. SQLite Official Website. Retrieved from <https://www.sqlite.org/about.html>

- [3] NuGet Gallery. (2024). BCrypt.Net-Next Package. NuGet Package Manager. Retrieved from <https://www.nuget.org/packages/BCrypt.Net-Next/>

- [4] Microsoft Corporation. (2024). Microsoft.Data.Sqlite Overview. Microsoft Documentation. Retrieved from <https://docs.microsoft.com/en-us/dotnet/standard/data/sqlite/>

- [5] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. Repository Pattern - Data Access Layer Design.

- [6] Freeman, A. (2020). Pro ASP.NET Core 3: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages. Apress. Chapter on Repository Pattern and Data Access.